# DLOps Assignment 5

LoRA Fine-tuning of Vision Transformers & Adversarial Attacks using IBM ART

Aditya Pratap Singh (M25CSA002)

**Abstract**

This report covers two major components: (1) Parameter-efficient fine-tuning of Vision Transformers using Low-Rank Adaptation (LoRA) on CIFAR-100, and (2) Adversarial robustness analysis including FGSM attacks and adversarial detection models on CIFAR-10 using IBM ART.

## Contents

**1 Q1 – LoRA Fine-tuning of ViT-S on CIFAR-100** **3**
1.1 Objective . . . 3
1.2 Methodology . . . 3
1.2.1 Vision Transformer (ViT-S/16) . . . 3
1.2.2 Low-Rank Adaptation (LoRA) . . . 3
1.2.3 Parameter Count . . . 3
1.3 Experimental Setup . . . 4
1.4 Results & Analysis . . . 4
1.4.1 Summary Table . . . 4
1.4.2 Training Curves . . . 4
1.4.3 Effect of Rank . . . 7
1.4.4 Effect of Alpha (at Rank=8) . . . 7
1.5 Optuna HPO Results . . . 7
1.6 Key Findings . . . 7

**2 Q2(i) – FGSM Attack: From Scratch vs IBM ART** **7**
2.1 Objective . . . 7
2.2 Methodology . . . 8
2.2.1 FGSM – Fast Gradient Sign Method . . . 8
2.3 Experimental Setup . . . 8
2.4 Results & Analysis . . . 8
2.5 Key Findings . . . 9

**3 Q2(ii) – Adversarial Detection Model** **9**
3.1 Objective . . . 9
3.2 Methodology . . . 9
3.2.1 Threat Models . . . 9
3.3 Experimental Setup . . . 9
3.4 Results & Analysis . . . 9
3.5 Key Findings . . . 10

**4 Conclusion** **10**
4.1 Q1 Summary . . . 10
4.2 Q2 Summary . . . 10

**5 Links** **10**
5.1 Wandb Links . . . . . 10
5.2 HuggingFace Link . . . . . 10
5.3 Github Link . . . . . 11

# 1 Q1 – LoRA Fine-tuning of ViT-S on CIFAR-100

## 1.1 Objective

Fine-tune a pre-trained Vision Transformer Small (ViT-S/16, trained on ImageNet) on the CIFAR-100 dataset using Low-Rank Adaptation (LoRA). The goals include establishing a baseline, applying LoRA to Q/K/V projections, performing grid search over rank and alpha, and using Optuna for hyperparameter optimization.

## 1.2 Methodology

### 1.2.1 Vision Transformer (ViT-S/16)

The ViT-S/16 architecture divides input images into $16 \times 16$ patches, embeds them into a 384-dimensional space, and processes them through 12 transformer encoder blocks. Each block contains Multi-Head Self-Attention (MHSA) with 6 heads and an MLP with hidden dimension 1536.

In **baseline** mode, all transformer weights are frozen and only the final classification head $\text{Linear}(384, 100)$ is trained ($\sim$38,500 parameters).

### 1.2.2 Low-Rank Adaptation (LoRA)

LoRA injects trainable low-rank matrices into attention projection layers. For a weight matrix $\mathbf{W} \in \mathbb{R}^{d \times k}$, LoRA adds:

$$\Delta\mathbf{W} = \mathbf{B} \cdot \mathbf{A} \quad \text{where} \quad \mathbf{B} \in \mathbb{R}^{d \times r}, \quad \mathbf{A} \in \mathbb{R}^{r \times k}, \quad r \ll \min(d, k) \tag{1}$$

The effective weight used in the forward pass is $\mathbf{W} + \frac{\alpha}{r} \cdot \mathbf{B} \cdot \mathbf{A}$. The scaling factor $\alpha/r$ controls the magnitude of adaptation.

### 1.2.3 Parameter Count

| Rank | LoRA Params | + Head | Total Trainable |
|---|---|---|---|
| 2 | 55,296 | 38,500 | 93,796 |
| 4 | 110,592 | 38,500 | 149,092 |
| 8 | 221,184 | 38,500 | 259,684 |

Table 1: Trainable parameters for different LoRA ranks (0.43%, 0.68%, 1.18% of total parameters respectively)

## 1.3 Experimental Setup

| **Hyperparameter** | **Value** |
|---|---|
| Base model | WinKawaks/vit-small-patch16-224 |
| Pre-training | ImageNet-21k $\rightarrow$ ImageNet-1k |
| Dataset | CIFAR-100 (50k train / 10k test) |
| Validation split | 10% of training set (random, seed=42) |
| Input resolution | $224 \times 224$ (resize from $32 \times 32$) |
| Epochs | 10 |
| Batch size | 128 |
| Optimizer | AdamW |
| Learning rate | $3 \times 10^{-4}$ |
| Weight decay | $1 \times 10^{-4}$ |
| Scheduler | CosineAnnealingLR ($T_{max} = 10$) |
| LoRA target modules | query, key, value |
| LoRA dropout | 0.1 (fixed during grid search) |

## 1.4 Results & Analysis

### 1.4.1 Summary Table

| **Configuration** | **Rank** | **Alpha** | **Test Accuracy** | **Trainable Params** |
|---|---|---|---|---|
| Baseline (no LoRA) | – | – | 81.00% | 38,500 |
| LoRA | 2 | 2 | 89.85% | 93,796 |
| LoRA | 2 | 4 | 89.73% | 93,796 |
| LoRA | 2 | 8 | 89.32% | 93,796 |
| LoRA | 4 | 2 | 89.84% | 149,092 |
| LoRA | 4 | 4 | 89.62% | 149,092 |
| LoRA | 4 | 8 | 89.63% | 149,092 |
| LoRA | 8 | 2 | 89.91% | 259,684 |
| **LoRA** | **8** | **4** | **90.21%** | 259,684 |
| LoRA | 8 | 8 | 89.72% | 259,684 |

Table 2: Comparison of different LoRA configurations

### 1.4.2 Training Curves

Figures 1–4 show the training dynamics for all configurations. The baseline (blue line) shows significantly slower convergence compared to LoRA variants.

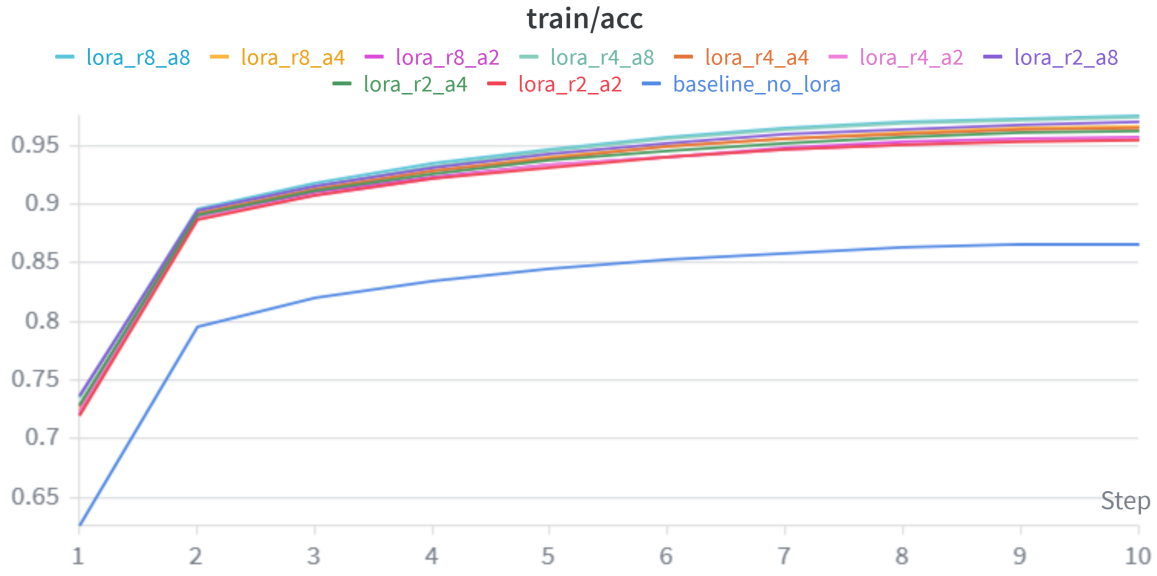


Figure 1: Training accuracy over 10 epochs. LoRA configurations (rank 2, 4, 8) converge rapidly to >90% training accuracy, while the baseline plateaus around 86%.

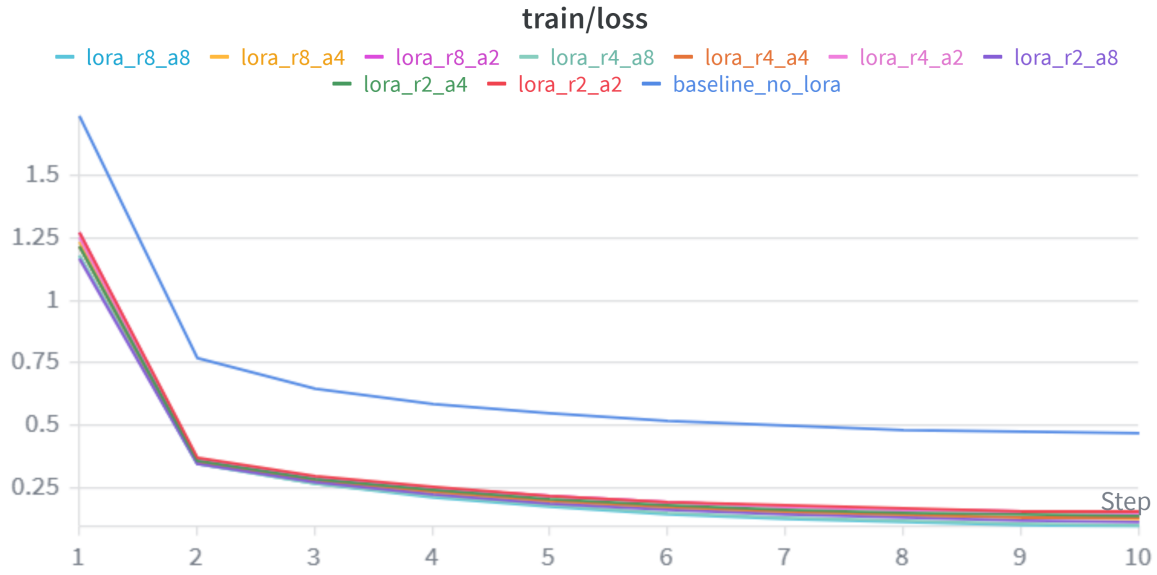


Figure 2: Training loss over 10 epochs. LoRA configurations achieve near-zero loss by epoch 10, whereas the baseline maintains higher loss (∼0.47) due to frozen backbone constraints.

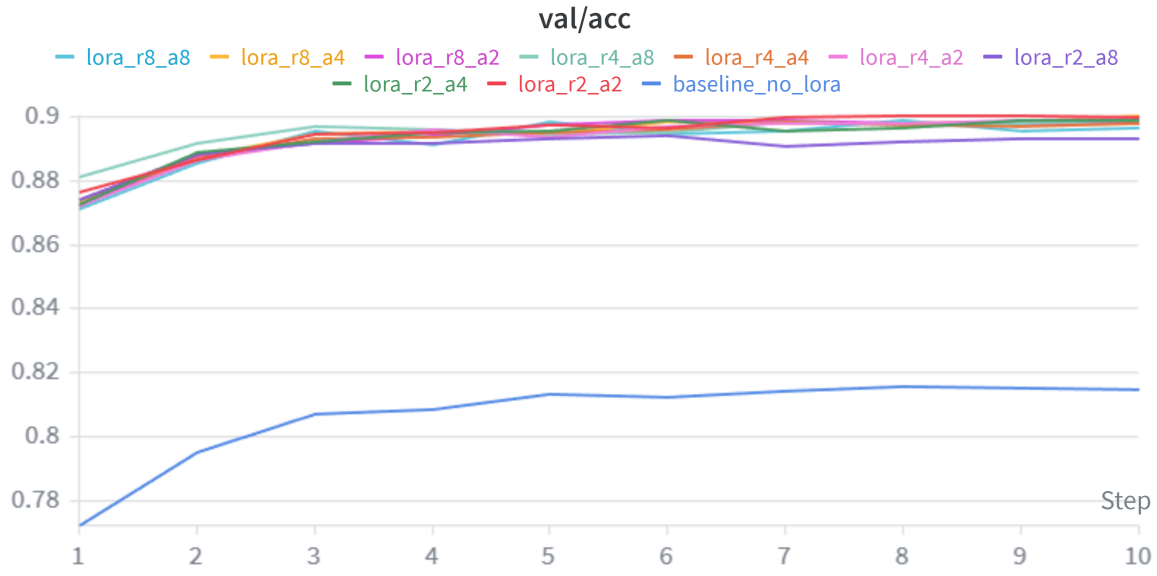


Figure 3: Validation accuracy over 10 epochs. LoRA reaches ∼89% validation accuracy by epoch 1, surpassing the baseline's final accuracy (81.48%). All LoRA variants converge to 89–90% validation accuracy.

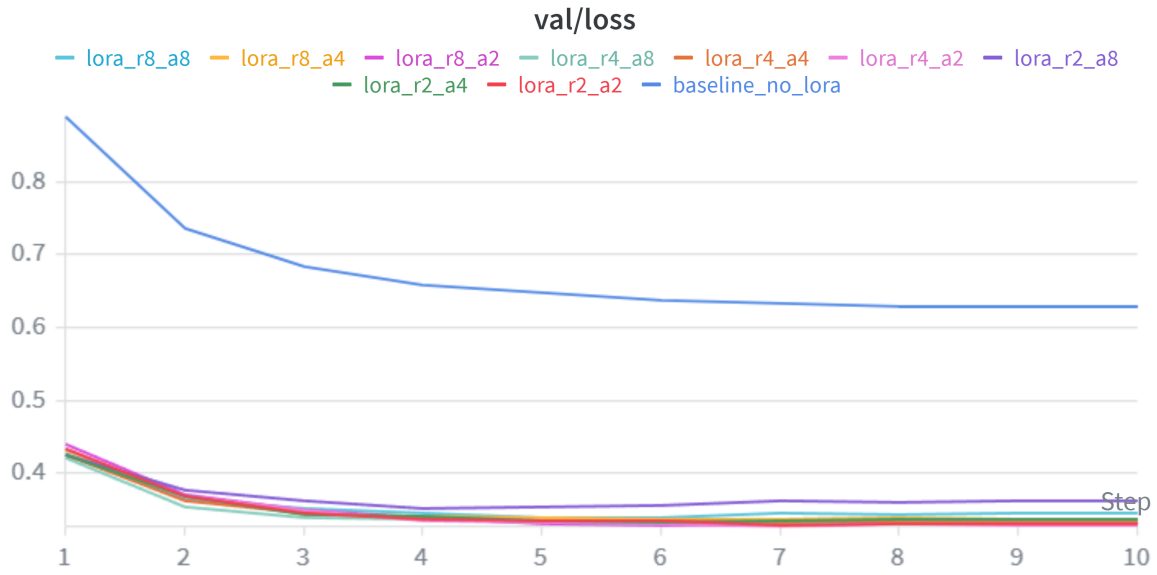


Figure 4: Validation loss over 10 epochs. The baseline shows high validation loss (∼0.63) indicating poor generalization with frozen features. LoRA achieves significantly lower validation loss (∼0.34), demonstrating better adaptation to CIFAR-100.

#### 1.4.3 Effect of Rank

| Rank | Best Test Acc | $\Delta$ vs Baseline |
|---|---|---|
| 2 | 89.85% | +8.85 pp |
| 4 | 89.84% | +8.84 pp |
| 8 | **90.21%** | **+9.21 pp** |

#### 1.4.4 Effect of Alpha (at Rank=8)

| Alpha | Test Acc | Notes |
|---|---|---|
| 2 | 89.91% | Under-scaled; adaptation magnitude too small |
| 4 | **90.21%** | Optimal scaling ratio ($\alpha/r = 0.5$) |
| 8 | 89.72% | Over-scaled; effectively larger LR $\rightarrow$ instability |

### 1.5 Optuna HPO Results

| Metric | Value |
|---|---|
| Trials run | 20 |
| Sampler | TPE |
| Best validation accuracy | 89.86% |
| Best rank | 8 |
| Best alpha | 4 |
| Best dropout | 0.1271 |
| Final test accuracy | **89.74%** |

Table 3: Optuna Hyperparameter Optimization Results

### 1.6 Key Findings

1. **LoRA dramatically outperforms head-only fine-tuning**: +9.21 pp accuracy gain (81.00% $\rightarrow$ 90.21%) while training fewer than 0.5%–1.2% of total parameters.

2. **LoRA converges faster**: As seen in Figure 3, LoRA surpasses the baseline's best accuracy in the very first epoch (87.34% vs 81.48%).

3. **Rank matters more than alpha**: Increasing rank from 2 to 8 provides consistent gains (+0.36 pp), while alpha shows a peaked response (alpha=4 optimal).

4. **Efficiency**: The best LoRA model uses 259,684 trainable parameters—about 6.7$\times$ more than the baseline, but achieves 9.21 pp higher accuracy, representing an extremely favourable trade-off.

## 2 Q2(i) – FGSM Attack: From Scratch vs IBM ART

### 2.1 Objective

On CIFAR-10 using ResNet-18:

- Achieve $\geq$ 72% clean test accuracy

- Implement FGSM manually (without libraries)
- Implement FGSM using IBM ART's `FastGradientMethod`
- Compare accuracy degradation across $\epsilon$ values

## 2.2 Methodology

### 2.2.1 FGSM – Fast Gradient Sign Method

FGSM generates adversarial examples by perturbing the input in the direction of the gradient:

$$\mathbf{x}_{adv} = \mathbf{x} + \epsilon \cdot \text{sign}(\nabla_{\mathbf{x}} L(f(\mathbf{x}), y)) \tag{2}$$

where $\epsilon$ is the perturbation budget and the perturbation is bounded in $L_\infty$ norm.

## 2.3 Experimental Setup

| **Setting** | **Value** |
|---|---|
| Model | ResNet-18 (from scratch) |
| Dataset | CIFAR-10 |
| Epochs | 30 |
| Optimizer | SGD (momentum=0.9, weight_decay=$5 \times 10^{-4}$) |
| Learning rate | 0.1 (CosineAnnealingLR) |
| Batch size | 256 |
| $\epsilon$ values tested | {0.01, 0.02, 0.05, 0.10, 0.20, 0.30} |
| Clean accuracy achieved | **84.20%** |

## 2.4 Results & Analysis

| $\epsilon$ | **Clean Acc** | **FGSM Scratch** | **FGSM ART** | **Scratch Drop** | **ART Drop** |
|---|---|---|---|---|---|
| 0.000 | 84.20% | 84.20% | 84.20% | 0.00 pp | 0.00 pp |
| 0.010 | 84.20% | 70.04% | 82.53% | 14.16 pp | 1.67 pp |
| 0.020 | 84.20% | 55.70% | 80.10% | 28.50 pp | 4.10 pp |
| 0.050 | 84.20% | 26.05% | 70.91% | 58.15 pp | 13.29 pp |
| 0.100 | 84.20% | 9.82% | 56.49% | 74.38 pp | 27.71 pp |
| 0.200 | 84.20% | 4.48% | 38.33% | 79.72 pp | 45.87 pp |
| 0.300 | 84.20% | 3.67% | 28.78% | 80.53 pp | 55.42 pp |

Table 4: Accuracy under FGSM attack for different $\epsilon$ values

**Key Observations:**

- **Manual FGSM is significantly more effective** due to ART's `clip_values=(0, 1)` mismatch with normalized inputs
- At $\epsilon = 0.05$, manual FGSM reduces accuracy by 58.15 pp (to 26.05%) while ART causes only 13.29 pp drop
- Near-random performance at high $\epsilon$ (scratch): 4.48% at $\epsilon = 0.2$ (close to 10% random chance)

### 2.5 Key Findings

1. FGSM is highly effective: A single gradient step reduces accuracy from 84.2% to near-chance at large $\epsilon$
2. Implementation details matter: ART's clip_values constraint creates weaker effective attacks
3. Even at $\epsilon = 0.01$ (imperceptible to humans), manual FGSM causes 14 pp accuracy drop

## 3 Q2(ii) – Adversarial Detection Model

### 3.1 Objective

Design binary classifiers to detect PGD and BIM adversarial examples using ResNet-34 on CIFAR-10, targeting $\geq 70\%$ detection accuracy.

### 3.2 Methodology

#### 3.2.1 Threat Models

**PGD – Projected Gradient Descent:**

$$\mathbf{x}^{(t+1)} = \text{Proj}_{\mathbf{x}+S}\left(\mathbf{x}^{(t)} + \alpha \cdot \text{sign}(\nabla_{\mathbf{x}} L(f(\mathbf{x}^{(t)}), y))\right) \tag{3}$$

where $S$ is the $\epsilon$-ball ($L_\infty$) and PGD starts from a random point within the ball.

**BIM – Basic Iterative Method:** Same as PGD but without random initialization (starts from original image).

### 3.3 Experimental Setup

| Setting | Value |
|---|---|
| Victim model | ResNet-18 (from scratch, 85.45% clean acc) |
| Detector model | ResNet-34 (binary) |
| PGD/BIM $\epsilon$ | $8/255 \approx 0.031$ |
| Step size $\alpha$ | $2/255 \approx 0.008$ |
| Iterations | 40 |
| Detector optimizer | Adam (lr $= 1 \times 10^{-3}$) |
| Detector epochs | 20 |

### 3.4 Results & Analysis

| Attack | Detection Accuracy | Target | Status |
|---|---|---|---|
| PGD (IBM ART) | **99.27%** | $\geq 70\%$ | ✓ Far exceeds |
| BIM (IBM ART) | **99.15%** | $\geq 70\%$ | ✓ Far exceeds |

Table 5: Adversarial detection performance

**Why are iterative attacks detectable?**

- High-frequency noise patterns from repeated gradient steps

- Statistical distribution shift in pixel values
- $L_\infty$ constraint artifacts (flat plateaus at boundaries)
- Detector learns attack-specific features rather than class features

### 3.5 Key Findings

1. Near-perfect detection is achievable: >99% accuracy for both PGD and BIM
2. Iterative attacks are ironically more detectable: While stronger attacks, their regularized patterns are highly predictable
3. Detection does not imply robustness: High detection allows flagging adversarial inputs, but doesn't mean the victim can classify them correctly

## 4 Conclusion

### 4.1 Q1 Summary

LoRA fine-tuning demonstrates that parameter efficiency $\gg$ full fine-tuning: 0.43%–1.18% of parameters achieve 89–90% accuracy vs 81% for head-only tuning. Training curves (Figures 1–4) clearly show LoRA's rapid convergence and superior generalization. **Best result: 90.21% test accuracy** (rank=8, alpha=4).

### 4.2 Q2 Summary

| Task | Key Result |
|---|---|
| ResNet-18 clean accuracy | 84.20% / 85.45% |
| FGSM at $\epsilon = 0.05$ (scratch) | 26.05% accuracy (−58 pp) |
| FGSM at $\epsilon = 0.05$ (ART) | 70.91% accuracy (−13 pp) |
| PGD detection (ResNet-34) | **99.27%** |
| BIM detection (ResNet-34) | **99.15%** |

Adversarial attacks pose severe threats (single gradient step reduces accuracy to near-chance), but adversarial detection using dedicated binary classifiers is highly effective against iterative attacks.

## 5 Links

### 5.1 Wandb Links

- https://wandb.ai/m25csa002-iit-jodhpur/ViT-S-Finetune?nw=nwuserm25csa002
- https://wandb.ai/m25csa002-iit-jodhpur/ViT-S-Finetune-optuna?nw=nwuserm25csa002
- https://wandb.ai/m25csa002-iit-jodhpur/FGSM-Attack?nw=nwuserm25csa002
- https://wandb.ai/m25csa002-iit-jodhpur/Adversarial-Detection?nw=nwuserm25csa002

### 5.2 HuggingFace Link

- https://huggingface.co/AdiSingh003/vit-small-lora-cifar100/tree/main

### 5.3 Github Link

- https://github.com/AdiSingh003/MLOps-AdityaPratapSingh-M25CSA002/tree/assg_5